SPFx Launcher Plan: Pluggable Framework Architecture

Document Status: Preview
Last Updated: October 1, 2026

Table of Contents

1. Executive Summary
2. Overview: Single SPFx App with Pluggable Add-Ins
3. Architecture & Design Philosophy
4. Access & Integration Points
    - 4.1 SharePoint Site Access
    - 4.2 Entra (Azure AD) Access
    - 4.3 Power Platform Access
5. UI Framework: Web Parts as Core Components
6. Recommended JavaScript Libraries
    - 6.1 Core Framework Libraries
    - 6.2 UI/Animation Libraries
    - 6.3 State Management & Events
    - 6.4 Interactive Effects Libraries
7. SPFx Supported JavaScript Libraries Reference
8. Partial Page Updates & React Event Signals
9. Multi-Threading & Performance
    - 9.1 TypeScript & JavaScript Threading Model
    - 9.2 Web Workers for Background Processing
    - 9.3 Async/Await Patterns
10. Separation of Concerns: MVC in SPFx
11. Authentication & Multi-Account Management
    - 11.1 Browser-Based Authentication
    - 11.2 Switching Between Regular & Admin Accounts
    - 11.3 Token Management
12. Implementation Roadmap
13. Technical Cheat Sheet
14. Glossary of Terms
15. Helpful Resources & URLs

Executive Summary

This document outlines a strategy for creating a **single, extensible SPFx web part application** that acts as a "container" or "launcher" for multiple pluggable add-ins. Rather than deploying separate SPFx solutions for each feature, this approach centralizes the framework while allowing easy creation of add-in modules without requiring Yeoman, Heft, or npm redeploy cycles.

Key Benefits:

- Single deployment of core framework
- Easy add-in creation without build tooling complexity
- Shared authentication, state, and execution context
- Access to SharePoint, Entra, and Power Platform resources
- Rich interactive UI with modern web parts and advanced styling
- Clear separation of concerns and code reusability

Use Cases: Internal tools, productivity dashboards, admin panels, content management interfaces, and multi-tenant business applications.

↑ Back to Top

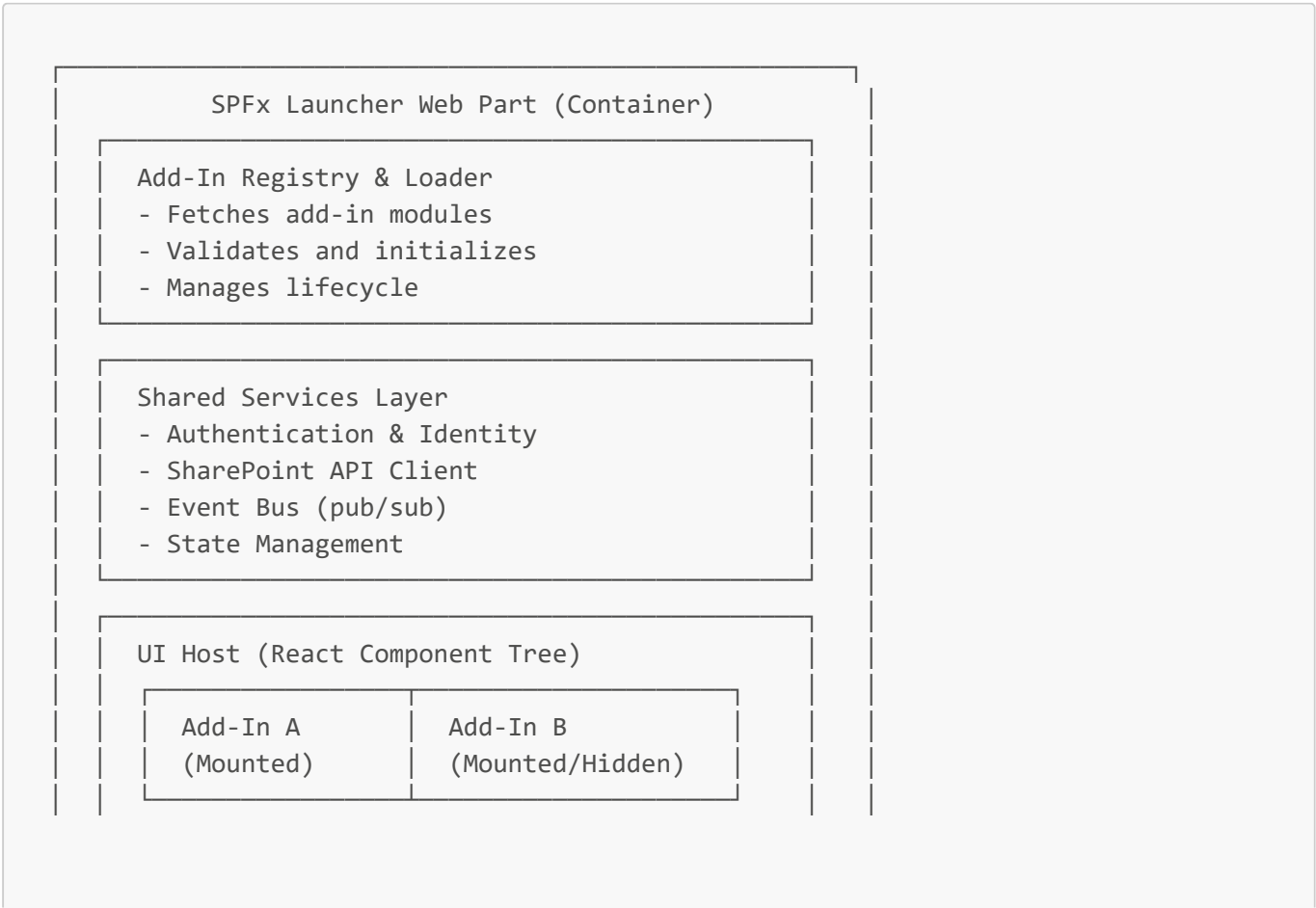
Overview: Single SPFx App with Pluggable Add-Ins

↑ Back to Top

The core concept is simple: deploy **one robust SPFx web part** that serves as the runtime container. Add-ins are:

- **Lightweight modules** (JavaScript/TypeScript bundles)
- **Stored in SharePoint** (document library, list items, or blob storage)
- **Dynamically loaded at runtime** (no redeploy needed)
- **Self-contained** with their own UI and logic
- **Integrated** with shared services and security context

Architecture Pattern



```
│  └──────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────┘
```

↑ Back to Top

Architecture & Design Philosophy

↑ Back to Top

1. Single Deployment Model

- One SPFx package deployed to tenant app catalog
- Framework is immutable (or versioned carefully)
- Add-ins deployed separately as asset files

2. Zero Build Complexity for Add-Ins

- Add-ins are pure JavaScript/TypeScript (no SPFx toolchain)
- Use lightweight bundlers: **Esbuild** or **Vite** (not Webpack)
- Developers write simple TypeScript, get .js output
- No Yeoman generator required

3. Shared Execution Context

- Common React context for all add-ins
- Single authentication session
- Shared API clients and configuration
- Event bus for inter-add-in communication

4. Modern Web Parts & UI Components

- Leverage Fluent UI (Office UI Fabric) for consistency
- Support drag-and-drop via react-beautiful-dnd or dnd-kit
- CSS-in-JS for interactive styling (emotion, styled-components)
- Web Animations API + GSAP for 3D/advanced effects

↑ Back to Top

Access & Integration Points

↑ Back to Top

4.1 SharePoint Site Access

↑ Back to Top

Method:

- Use **MSGraphClientFactory** (built into SPFx context) to call Microsoft Graph API

- Authenticate via user's existing SharePoint session (no extra login)
- Read/write to lists, libraries, and site pages

Key APIs:

```
// From SPFx context
const graphClient = this.context.msGraphClientFactory.getClient();

// List items
GET /sites/{siteId}/lists/{listId}/items
POST /sites/{siteId}/lists/{listId}/items

// Document libraries
GET /sites/{siteId}/drive/items
POST /sites/{siteId}/drive/items/{itemId}/upload

// Site pages
GET /sites/{siteId}/pages
```

In Iframe/New Tab:

- For isolated admin experiences, open SharePoint admin center in new tab
- Use postMessage API to communicate back to parent window
- Add-in can trigger actions and listen for responses

↑ Back to Top

4.2 Entra (Azure AD) Access

↑ Back to Top

User Identity:

- SPFx context provides current user via `this.context.pageContext.user`
- Includes upn, displayName, email

Switching Accounts (Regular ↔ Admin):

1. Detect admin role via **Entra groups** or **SharePoint permissions**
2. Prompt user to switch account (opens Azure login in iframe/popup)
3. Use **MSAL (Microsoft Authentication Library)** for browser-based auth
4. Maintain separate tokens for regular and admin personas
5. Switch context via state management (Redux/Zustand)

Implementation:

```
import { PublicClientApplication } from "@azure/msal-browser";

const pca = new PublicClientApplication({
  auth: { clientId: "YOUR_APP_ID" },
```

```
});

// Admin token acquisition
const adminToken = await pca.acquireTokenPopup({
  scopes: ["https://management.azure.com/.default"],
});
```

Admin Context Switch:

- Store both regular and admin tokens in secure state
- Expose UI toggle or dropdown to switch personas
- Update API calls to use correct token based on context

↑ Back to Top

4.3 Power Platform Access

↑ Back to Top

Power Apps Environment Access:

- **Power Automate** for workflow automation
- **Power Apps** portal access via iframe
- **Dataverse** API for data operations

Methods:

1. **Direct Dataverse API Calls:**

   ```
   // Use OData endpoint
   GET https://{env}.dynamics.com/api/data/v9.2/accounts
   ```

2. **Power Automate Cloud Flows:**
   - Create HTTP-triggered flows
   - Call from add-in via HTTP request
   - Flow handles Dataverse/Power Apps operations
3. **Embedded Power Apps Canvas:**
   - Embed Power App in iframe within add-in
   - Use postMessage for parent ↔ child communication
   - Pass data, switch environments

Environment Management UI:

- Fetch available environments from Power Platform Admin API
- Display dropdown to switch environments
- Store environment URL in context for subsequent calls

↑ Back to Top

UI Framework: Web Parts as Core Components

↑ Back to Top

Each **add-in is essentially a React component** that gets mounted into the container web part's DOM. Add-ins should export:

```
// add-in-example/index.ts
export interface IAddInConfig {
  id: string;
  name: string;
  icon?: string;
  description?: string;
}

export interface IAddInProps {
  config: IAddInConfig;
  context: ISharedContext;  // Shared auth, API clients, events
  onClose?: () => void;
}

// The component
export const MyAddIn: React.FC<IAddInProps> = ({ context, config }) => {
  return (
    <div className="addon-container">
      {/* Your UI here */}
    </div>
  );
};

// Metadata
export const ADDON_CONFIG: IAddInConfig = {
  id: "my-addon",
  name: "My Add-In",
  icon: "icon-url",
  description: "Does something useful"
};
```

Shared Context Injection

```
export interface ISharedContext {
  user: IPageContextUser;
  graphClient: GraphHttpClient;
  spHttpClient: SPHttpClient;
  eventBus: EventEmitter;
  stateManager: StateManager;
  config: IFrameworkConfig;
```

```
  switchAccount: (accountType: 'regular' | 'admin') => Promise<void>;
}
```

Component Lifecycle

1. **Registry Discovery** - Container fetches add-in manifests
2. **Load** - Dynamically import add-in module
3. **Initialize** - Inject shared context
4. **Mount** - Render in designated container
5. **Unmount** - Clean up event listeners and timers
6. **Update** - React re-renders on state/prop changes

↑ Back to Top

Recommended JavaScript Libraries

↑ Back to Top

6.1 Core Framework Libraries

↑ Back to Top

Library	Purpose	SPFx Support	Notes
React	UI framework	✅ Yes	Required by SPFx
TypeScript	Type safety	✅ Yes	Recommended
Fluent UI (Office UI Fabric)	Microsoft UI components	✅ Yes	Use `@fluentui/react`
@pnp/sp	SharePoint API wrapper	✅ Yes	Abstracts REST calls
@pnp/graph	Microsoft Graph wrapper	✅ Yes	Fluent Graph API
MSAL (Azure)	Authentication	✅ Yes	Multi-account support

↑ Back to Top

6.2 UI/Animation Libraries

↑ Back to Top

Library	Purpose	SPFx Support	Notes
Framer Motion	React animations	✅ Yes	Declarative motion
GSAP	Advanced animations & 3D	✅ Yes	Industry standard for complex effects
Three.js	3D graphics	✅ Yes	WebGL-based 3D engine

Library	Purpose	SPFx Support	Notes
Babylon.js	3D framework	✅ Yes	Microsoft-backed alternative
Tailwind CSS	Utility-first CSS	✅ Yes	Lightweight, responsive design
emotion / styled-components	CSS-in-JS	✅ Yes	Dynamic styling

↑ Back to Top

6.3 State Management & Events

↑ Back to Top

Library	Purpose	SPFx Support	Notes
Redux	Global state	✅ Yes	Industry standard, verbose
Zustand	Lightweight state	✅ Yes	Simpler alternative
Jotai	Atomic state	✅ Yes	Composable atoms
EventEmitter2	Pub/Sub messaging	✅ Yes	Inter-component communication
RxJS	Reactive programming	✅ Yes	Observable streams

↑ Back to Top

6.4 Interactive Effects Libraries

↑ Back to Top

Library	Purpose	SPFx Support	Notes
react-beautiful-dnd	Drag and drop	✅ Yes	Easy drag-drop lists
dnd-kit	Modern drag-drop	✅ Yes	Newer, more flexible
Sortable.js	List reordering	✅ Yes	Pure JS, framework-agnostic
react-dnd	Advanced drag-drop	✅ Yes	Maximum flexibility
Wavesurfer.js	Audio visualization	✅ Yes	For media add-ins
Chart.js / Recharts	Data visualization	✅ Yes	Charts and graphs

↑ Back to Top

SPFx Supported JavaScript Libraries Reference

↑ Back to Top

This table comprehensively lists JavaScript libraries that have been tested and confirmed to work with SPFx web parts:

Category	Library	Version	Status	Bundle Size	Notes
React Ecosystem	react	16.x - 18.x	✅ Supported	~40 KB	Core framework
	react-dom	16.x - 18.x	✅ Supported	~35 KB	DOM rendering
	react-router-dom	6.x	✅ Supported	~50 KB	Client-side routing
	react-query	3.x - 5.x	✅ Supported	~30 KB	Server state management
	react-hook-form	7.x	✅ Supported	~9 KB	Form handling
UI Frameworks	@fluentui/react	8.x - 9.x	✅ Supported	~200 KB	Microsoft UI components
	@fluentui/react-components	9.x	✅ Supported	~300 KB	Newer Fluent v9
	Material-UI	5.x	✅ Supported	~150 KB	Material Design
	Ant Design	4.x - 5.x	✅ Supported	~180 KB	Enterprise UI
	Bootstrap	5.x	✅ Supported	~50 KB	CSS framework
State Management	Redux	4.x	✅ Supported	~60 KB	Predictable state
	Redux Toolkit	1.x	✅ Supported	~65 KB	Simplified Redux
	Zustand	4.x	✅ Supported	~2 KB	Lightweight state
	Jotai	2.x	✅ Supported	~4 KB	Atomic state
	Recoil	0.7.x	✅ Supported	~20 KB	Facebook's state lib
	MobX	6.x	✅ Supported	~25 KB	Reactive programming

Category	Library	Version	Status	Bundle Size	Notes
Data & APIs	@pnp/sp	3.x - 4.x	✅ Supported	~50 KB	SharePoint API
	@pnp/graph	3.x - 4.x	✅ Supported	~40 KB	Microsoft Graph API
	axios	1.x	✅ Supported	~15 KB	HTTP client
	fetch-intercept	1.x	✅ Supported	~2 KB	Fetch polyfill
	graphql / graphql-request	15.x / 5.x	✅ Supported	~30 KB	GraphQL client
Animation	framer-motion	10.x	✅ Supported	~50 KB	React animations
	react-spring	9.x	✅ Supported	~20 KB	Spring physics animations
	GSAP	3.x	✅ Supported	~200 KB	Professional animations
	Three.js	140+x	✅ Supported	~600 KB	3D graphics
	Babylon.js	5.x - 6.x	✅ Supported	~800 KB	Microsoft 3D engine
Drag & Drop	react-beautiful-dnd	13.x	✅ Supported	~60 KB	List drag-drop
	dnd-kit	8.x	✅ Supported	~40 KB	Modern drag-drop
	react-dnd	16.x	✅ Supported	~80 KB	Advanced drag-drop
Utilities	lodash-es	4.x	✅ Supported	~70 KB	Utility functions
	date-fns	2.x	✅ Supported	~13 KB	Date manipulation
	uuid	9.x	✅ Supported	~1 KB	Generate UUIDs
	validator.js	13.x	✅ Supported	~50 KB	Data validation

Category	Library	Version	Status	Bundle Size	Notes
Authentication	@azure/msal-browser	2.x - 3.x	✅ Supported	~100 KB	Azure AD authentication
	auth0-js	9.x	✅ Supported	~50 KB	Auth0 integration
Testing	Jest	29.x	✅ Supported	Dev-only	Unit testing
	React Testing Library	14.x	✅ Supported	Dev-only	Component testing

Notes:

- Most libraries work well with SPFx if bundled correctly
- Always test third-party packages in your tenant first
- Some older IE11 support may require polyfills
- Bundle size monitoring is important for add-in performance

↑ Back to Top

Partial Page Updates & React Event Signals

↑ Back to Top

React-Based Event System

Use a **centralized event bus** to coordinate updates across add-ins without full page refresh:

```
// services/eventBus.ts
import { EventEmitter } from 'events';

export interface PageUpdateEvent {
  type: 'data-changed' | 'selection-changed' | 'config-updated';
  addInId: string;
  payload: any;
  timestamp: number;
}

export class EventBus extends EventEmitter {
  publishUpdate(event: PageUpdateEvent) {
    this.emit(event.type, event);
  }

  subscribe(eventType: string, callback: (e: PageUpdateEvent) => void) {
    this.on(eventType, callback);
    return () => this.off(eventType, callback); // Unsubscribe function
```

```
  }
}
```

Add-In Integration Example

```
import { useEffect, useState } from 'react';

export const MyAddIn: React.FC<IAddInProps> = ({ context }) => {
  const [data, setData] = useState(null);

  useEffect(() => {
    // Subscribe to updates from other add-ins
    const unsubscribe = context.eventBus.subscribe('data-changed', (event) => {
      if (event.addInId !== 'my-addon') {
        setData(event.payload); // Trigger re-render
      }
    });

    return unsubscribe; // Cleanup
  }, [context.eventBus]);

  const handleDataUpdate = (newData) => {
    setData(newData);
    // Publish event for other add-ins to react to
    context.eventBus.publishUpdate({
      type: 'data-changed',
      addInId: 'my-addon',
      payload: newData,
      timestamp: Date.now()
    });
  };

  return <div>{/* Render data */}</div>;
};
```

Partial Updates with React.memo & useMemo

Prevent unnecessary re-renders:

```
import React, { useMemo, useCallback } from 'react';

const OptimizedChild = React.memo(({ item, onSelect }) => {
  return <div onClick={() => onSelect(item.id)}>{item.name}</div>;
});

export const ParentAddIn: React.FC = ({ context }) => {
  const [selectedId, setSelectedId] = useState(null);
  const [items, setItems] = useState([]);
```

```
  // Memoize expensive computation
  const filteredItems = useMemo(
    () => items.filter(i => i.active),
    [items]
  );

  // Memoize callback to prevent child re-renders
  const handleSelect = useCallback((id) => {
    setSelectedId(id);
  }, []);

  return (
    <div>
      {filteredItems.map(item => (
        <OptimizedChild key={item.id} item={item} onSelect={handleSelect} />
      ))}
    </div>
  );
};
```

↑ Back to Top

Multi-Threading & Performance

↑ Back to Top

9.1 TypeScript & JavaScript Threading Model

↑ Back to Top

Single-Threaded Reality:

- JavaScript (and TypeScript) runs in a **single thread** by default
- No true multi-threading like C# or Java
- Event loop handles asynchronous operations
- Long-running tasks block the UI thread

Event Loop Phases:

1. **Microtasks** (Promises, queueMicrotask)
2. **Rendering** (if needed)
3. **Macrotasks** (setTimeout, setInterval, events)

Common Blocking Scenarios:

- Heavy computation (parsing large JSON, image processing)
- Large DOM operations
- Complex calculations

↑ Back to Top

9.2 Web Workers for Background Processing

↑ Back to Top

Web Workers enable true parallel execution:

```
// worker.ts (runs in separate thread)
self.onmessage = (event: MessageEvent) => {
  const { data } = event;

  // Heavy computation on separate thread
  const result = complexCalculation(data);

  // Send result back to main thread
  self.postMessage({ result });
};

function complexCalculation(input: any[]): any {
  // Process large dataset, image, etc.
  return input.map(/* ... */);
}
```

```
// addIn component (main thread)
export const MyAddIn: React.FC<IAddInProps> = () => {
  const [result, setResult] = useState(null);
  const workerRef = useRef<Worker | null>(null);

  useEffect(() => {
    // Create worker
    workerRef.current = new Worker(new URL('./worker.ts', import.meta.url), {
      type: 'module',
    });

    // Listen for results
    workerRef.current.onmessage = (event) => {
      setResult(event.data.result);
    };

    return () => workerRef.current?.terminate();
  }, []);

  const handleProcessData = (largeDataset: any[]) => {
    workerRef.current?.postMessage({ data: largeDataset });
  };

  return <button onClick={() => handleProcessData(data)}>Process</button>;
};
```

Use Cases for Workers:

- Image processing/filters
- Large data transformations
- Compression/decompression
- PDF generation
- Cryptography
- Real-time data analysis

↑ Back to Top

9.3 Async/Await Patterns

↑ Back to Top

Async/await doesn't create new threads but prevents blocking:

```
// ❌ Blocks UI
async function fetchDataBlocking() {
  const response = await fetch('/api/data');
  const data = await response.json();
  // UI is unresponsive until complete
  return data;
}

// ✅ Non-blocking with loading state
export const DataFetchAddIn: React.FC = () => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(false);

  const fetchData = async () => {
    setLoading(true);
    try {
      const response = await fetch('/api/data');
      const result = await response.json();
      setData(result);
      // UI remains responsive, can update progress
    } finally {
      setLoading(false);
    }
  };

  return (
    <>
      <button onClick={fetchData} disabled={loading}>
        {loading ? 'Loading...' : 'Fetch Data'}
      </button>
      {data && <pre>{JSON.stringify(data, null, 2)}</pre>}
    </>
  );
};
```

Request Animation Frame (RAF) for Smooth Updates:

```
// Progressive rendering without jank
export const ProgressiveLoadAddIn: React.FC = () => {
  const [items, setItems] = useState([]);

  const loadItemsProgressively = async (largeList: any[]) => {
    let index = 0;

    const processNextBatch = () => {
      const batch = largeList.slice(index, (index += 50));
      setItems(prev => [...prev, ...batch]);

      if (index < largeList.length) {
        requestAnimationFrame(processNextBatch);
      }
    };

    requestAnimationFrame(processNextBatch);
  };

  return <div>{/* render items */}</div>;
};
```

↑ Back to Top

Separation of Concerns: MVC in SPFx

↑ Back to Top

Adapting MVC Pattern to SPFx/React

Traditional MVC (Model-View-Controller) maps to modern React architecture:

```
MVC                    React/SPFx
──────────────────────────────────────────
Model      →  State/Redux/Zustand store + API layer
View       →  React Components (JSX/TSX)
Controller →  Custom hooks + Event handlers
```

Recommended Architecture: Model-View-ViewModel (MVVM)

```
// models/userModel.ts (Model)
export interface IUser {
  id: string;
  name: string;
  email: string;
  permissions: string[];
}
```

```
export class UserModel {
  constructor(private graphClient: GraphHttpClient) {}

  async fetchUser(userId: string): Promise<IUser> {
    const response = await this.graphClient.get(`/me`);
    return response;
  }

  async updateUser(user: IUser): Promise<IUser> {
    const response = await this.graphClient.patch(`/me`, user);
    return response;
  }
}

// viewModels/userViewModel.ts (ViewModel)
import { create } from 'zustand';

interface UserState {
  user: IUser | null;
  loading: boolean;
  error: string | null;
  fetchUser: (id: string) => Promise<void>;
  updateUser: (user: IUser) => Promise<void>;
}

export const useUserStore = create<UserState>((set) => ({
  user: null,
  loading: false,
  error: null,

  fetchUser: async (id: string) => {
    set({ loading: true });
    try {
      const user = await userModel.fetchUser(id);
      set({ user, error: null });
    } catch (error) {
      set({ error: error.message });
    } finally {
      set({ loading: false });
    }
  },

  updateUser: async (user: IUser) => {
    try {
      const updated = await userModel.updateUser(user);
      set({ user: updated });
    } catch (error) {
      set({ error: error.message });
    }
  }
}));

// views/UserDetailsView.tsx (View)
```

```
export const UserDetailsView: React.FC = () => {
  const { user, loading, error, fetchUser, updateUser } = useUserStore();

  useEffect(() => {
    fetchUser('me');
  }, []);

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;
  if (!user) return null;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
      {/* Update form */}
    </div>
  );
};
```

Layered Architecture for Add-Ins

```
┌─────────────────────────────────────────┐
│     Presentation (React Components)     │ ← Views
├─────────────────────────────────────────┤
│  Application (Custom Hooks, State)      │ ← ViewModels
├─────────────────────────────────────────┤
│   Domain (Business Logic, Models)       │ ← Models
├─────────────────────────────────────────┤
│  Infrastructure (API, Storage)          │ ← Data Access
└─────────────────────────────────────────┘
```

↑ Back to Top

Authentication & Multi-Account Management

↑ Back to Top

11.1 Browser-Based Authentication

↑ Back to Top

SPFx Built-in Context (No Extra Setup):

```
// User already authenticated via SharePoint session
const user = this.context.pageContext.user;
```

```
console.log(user.displayName); // Current user
console.log(user.loginName);   // UPN
```

For Additional Scopes (Admin APIs, Power Platform):

Use **MSAL (Microsoft Authentication Library)**:

```
import { PublicClientApplication, EventType } from "@azure/msal-browser";

const msalConfig = {
  auth: {
    clientId: "YOUR_APP_ID",
    authority: "https://login.microsoftonline.com/common",
    redirectUri: window.location.origin,
  },
  cache: {
    cacheLocation: "sessionStorage", // or localStorage
    storeAuthStateInCookie: false,
  },
};

export const pca = new PublicClientApplication(msalConfig);

// Initialize MSAL
await pca.initialize();

// Handle redirect if login was initiated
if (pca.getAccount() === null) {
  // No account signed in
} else {
  // Account already signed in
}
```

Acquiring Tokens:

```
// Silent token acquisition (cached)
const getToken = async (scope: string): Promise<string> => {
  try {
    const result = await pca.acquireTokenSilent({
      scopes: [scope],
      account: pca.getActiveAccount(),
    });
    return result.accessToken;
  } catch (error) {
    // Fall back to interactive login
    return pca.acquireTokenPopup({ scopes: [scope] })
      .then(result => result.accessToken);
  }
};
```

↑ Back to Top

11.2 Switching Between Regular & Admin Accounts

↑ Back to Top

Architecture:

```
export interface IAuthContext {
  currentAccount: 'regular' | 'admin';
  regularToken: string | null;
  adminToken: string | null;
  switchAccount: (account: 'regular' | 'admin') => Promise<void>;
  getActiveToken: () => string;
}

export const useAuthStore = create<IAuthContext>((set, get) => ({
  currentAccount: 'regular',
  regularToken: null,
  adminToken: null,

  switchAccount: async (account: 'regular' | 'admin') => {
    set({ currentAccount: account });
    // Update API clients to use correct token
  },

  getActiveToken: () => {
    const state = get();
    return state.currentAccount === 'admin'
      ? state.adminToken || ''
      : state.regularToken || '';
  }
}));
```

UI Component for Switching:

```
export const AccountSwitcher: React.FC = () => {
  const { currentAccount, switchAccount } = useAuthStore();

  const handleSwitch = async () => {
    const newAccount = currentAccount === 'admin' ? 'regular' : 'admin';
    await switchAccount(newAccount);
  };

  return (
    <button onClick={handleSwitch}>
      Currently logged in as: {currentAccount}
      {currentAccount === 'admin' && ' 👨‍💼 '}
      {currentAccount === 'regular' && ' 👤 '}
    </button>
  );
};
```

Detecting Admin Capability:

```
export const canAccessAdmin = async (context: ISharedContext): Promise<boolean> =>
{
  try {
    // Check if user is in admin group
    const response = await context.graphClient.get('/me/memberOf');
    const groups = response.value as any[];

    return groups.some(g =>
      g.displayName?.includes('Admin') ||
      g.id === 'ADMIN_GROUP_ID'
    );
  } catch {
    return false;
  }
};
```

↑ Back to Top

11.3 Token Management

↑ Back to Top

Secure Token Storage:

```
// Use sessionStorage for tokens (cleared on browser close)
export const tokenStore = {
  set: (key: string, token: string) => {
```

```
    sessionStorage.setItem(`auth_${key}`, token);
  },
  get: (key: string): string | null => {
    return sessionStorage.getItem(`auth_${key}`);
  },
  clear: (key: string) => {
    sessionStorage.removeItem(`auth_${key}`);
  }
};
```

Token Expiration Handling:

```
const isTokenExpired = (token: string): boolean => {
  try {
    // Decode JWT
    const payload = JSON.parse(
      atob(token.split('.')[1])
    );
    const expirationTime = payload.exp * 1000; // Convert to milliseconds
    return Date.now() >= expirationTime;
  } catch {
    return true;
  }
};

// Middleware for API calls
export const withTokenRefresh = async (
  fn: () => Promise<any>,
  getToken: () => Promise<string>
): Promise<any> => {
  try {
    return await fn();
  } catch (error: any) {
    if (error.status === 401) {
      // Token expired, refresh
      const newToken = await getToken();
      return fn();
    }
    throw error;
  }
};
```

↑ Back to Top

Implementation Roadmap

↑ Back to Top

Phase 1: Foundation (Weeks 1-2)

- [ ] Create SPFx web part project with React + TypeScript
- [ ] Set up Fluent UI and base styling
- [ ] Create shared context and event bus
- [ ] Implement add-in loader/registry
- [ ] Create authentication wrapper (MSAL integration)

Phase 2: Core Features (Weeks 3-4)

- [ ] SharePoint API client (via @pnp/sp)
- [ ] Power Platform integration (Dataverse, environments)
- [ ] Entra authentication and token management
- [ ] Admin account switching UI
- [ ] State management (Zustand store)

Phase 3: Add-In Framework (Weeks 5-6)

- [ ] Create add-in template/scaffold
- [ ] Build lightweight module loader
- [ ] Implement inter-add-in communication (event bus)
- [ ] Add partial page update mechanism
- [ ] Documentation and examples

Phase 4: UI Enhancements (Weeks 7-8)

- [ ] Drag-and-drop framework (dnd-kit)
- [ ] Animation library integration (Framer Motion, GSAP)
- [ ] Interactive CSS effects
- [ ] Performance optimization (React.memo, useMemo)
- [ ] Web Worker integration for heavy tasks

Phase 5: Testing & Deployment (Weeks 9-10)

- [ ] Unit tests (Jest + React Testing Library)
- [ ] Integration tests
- [ ] E2E tests (Cypress/Playwright)
- [ ] Security audit
- [ ] Package and deploy to app catalog

↑ Back to Top

Technical Cheat Sheet

↑ Back to Top

Quick Reference Commands

SPFx Project Setup:

```
# Create new SPFx project
yo @microsoft/sharepoint

# Build
npm run build

# Serve locally
npm run serve

# Package for production
npm run bundle --ship
```

Add-In Bundle (using Esbuild):

```
# Install esbuild
npm install -D esbuild

# Build add-in to single JS file
esbuild src/addIn.tsx --bundle --outfile=dist/addIn.js --loader:.ts=tsx --
external:react --external:react-dom
```

Essential Code Snippets

Minimal Add-In Template:

```
import React from 'react';
import { IAddInProps, IAddInConfig } from '../types';

export const MyAddIn: React.FC<IAddInProps> = ({ context, config }) => {
  return (
    <div className="addon-wrapper">
      <h2>{config.name}</h2>
      <button onClick={() => context.eventBus.publishUpdate({
        type: 'data-changed',
        addInId: config.id,
        payload: { message: 'Hello from add-in' },
        timestamp: Date.now()
      })}>
        Send Message
      </button>
    </div>
  );
};

export const ADDON_CONFIG: IAddInConfig = {
  id: 'my-addon',
  name: 'My Add-In',
```

```
  description: 'Sample add-in'
};
```

API Call with Token:

```
const callGraphAPI = async (
  endpoint: string,
  token: string
): Promise<any> => {
  const response = await fetch(
    `https://graph.microsoft.com/v1.0${endpoint}`,
    {
      headers: {
        Authorization: `Bearer ${token}`,
        'Content-Type': 'application/json'
      }
    }
  );

  if (!response.ok) {
    throw new Error(`Graph API error: ${response.status}`);
  }

  return response.json();
};
```

Event Bus Usage:

```
// Publish event
context.eventBus.publishUpdate({
  type: 'data-changed',
  addInId: 'my-addon',
  payload: { userId: 123 },
  timestamp: Date.now()
});

// Subscribe to event
const unsubscribe = context.eventBus.subscribe('data-changed', (event) => {
  console.log('Data changed:', event.payload);
});
```

↑ Back to Top

Glossary of Terms

↑ Back to Top

Term	Definition
Add-In / Plugin	Lightweight module that extends the main SPFx launcher application
App Catalog	SharePoint repository where SPFx solutions are deployed and managed
Dataverse	Microsoft's cloud-based data platform (part of Power Platform)
Entra (formerly Azure AD)	Microsoft's identity and access management service
Event Bus	Centralized pub/sub system for inter-component communication
Fluent UI	Microsoft's design system and React component library
Graph API	Microsoft's unified API for accessing Microsoft 365 data
MSAL	Microsoft Authentication Library for browser-based OAuth flows
Module Federation	Webpack feature allowing dynamic loading of application modules
PnP JS	Community library providing fluent API wrappers for SharePoint and Graph
SPFx	SharePoint Framework - Microsoft's extension model for modern SharePoint
Web Worker	Browser API enabling parallel execution of JavaScript
Yeoman	Scaffolding tool (part of traditional SPFx setup)
Zustand	Lightweight state management library for React

↑ Back to Top

Helpful Resources & URLs

↑ Back to Top

Microsoft Official Documentation

- **SharePoint Framework Docs**: https://learn.microsoft.com/en-us/sharepoint/dev/spfx/sharepoint-framework-overview
- **Microsoft Graph API**: https://learn.microsoft.com/en-us/graph/overview
- **MSAL Browser**: https://learn.microsoft.com/en-us/azure/active-directory/develop/msal-browser-migration-guide
- **Fluent UI React**: https://microsoft.github.io/fluentui-react/
- **PnP JavaScript Library**: https://pnp.github.io/pnpjs/

Community & Open Source

- **PnP Samples**: https://github.com/pnp/sp-dev-fx-webparts
- **Microsoft 365 Dev Program**: https://developer.microsoft.com/en-us/microsoft-365/dev-program
- **GitHub PnP Community**: https://github.com/orgs/pnp/repositories

Library Documentation

- **React**: https://react.dev
- **TypeScript**: https://www.typescriptlang.org/docs/
- **Zustand**: https://github.com/pmndrs/zustand
- **dnd-kit**: https://docs.dndkit.com/
- **Framer Motion**: https://www.framer.com/motion/
- **GSAP**: https://gsap.com/docs/
- **Three.js**: https://threejs.org/docs/
- **Web Workers**: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Tools & Utilities

- **Esbuild**: https://esbuild.github.io/
- **Vite**: https://vitejs.dev/
- **Webpack Module Federation**: https://webpack.js.org/concepts/module-federation/
- **Jest Testing**: https://jestjs.io/
- **Cypress E2E Testing**: https://www.cypress.io/

Performance & Best Practices

- **Web Vitals**: https://web.dev/vitals/
- **React Performance**: https://react.dev/learn/render-and-commit
- **Bundle Analysis**: https://webpack.js.org/plugins/bundle-analyzer/

End of Document

For questions, suggestions, or updates to this plan, please refer to your development team or documentation owner.